RedFlag

Cybersecurity Due Diligence Tool - Project Brief

Generated: 2026-05-22

1. What RedFlag Is

RedFlag is a multi-scanner cybersecurity due-diligence tool built for M&A assessments. It combines findings from multiple security scanners, normalises them into a shared internal schema, scores them using a weighted risk model, and presents the results through a Streamlit web UI with a downloadable CSV report.

The goal is not to produce disconnected scanner outputs. The goal is a single unified risk picture where every scanner contributes evidence to one common score.

2. Current Scanner Status

Each scanner goes through two integration stages:

- Stage A - Connectivity and data-display validation (prove it works, show raw output in UI)
- Stage B - Risk-scoring integration (merge signals into the central triage engine)

Nmap	*Stage A + B complete*	Primary scan engine. Produces open port / service findings. Fully wired into triage.
Shodan	*Stage A + B complete*	Host enrichment. Upgrades exposure and exploit status for internet-visible ports.
OpenVAS	*Not yet started*	Verified vulnerability scanner. Will provide confirmed CVEs and CVSS scores.
ZAP	*Not yet started*	Web application scanner. Will provide HTTP-layer findings.

3. Repository Structure

```
Redflag/
  app.py                    Streamlit frontend and orchestration
  config.py                 Configuration
  .env                      API keys (SHODAN_API_KEY etc.)

  scanners/
    nmap_scan.py            Runs nmap -sV --open, saves XML output
    shodan_scan.py          Shodan api.host() lookup + enrichment logic
    openvas_parse.py        Placeholder - not yet integrated
    zap_scan.py             Placeholder - not yet integrated

  analysis/
    schema.py               Finding data model (Pydantic) + all enums
    parser.py               Parses Nmap XML into Finding objects
    triage.py               Weighted risk scoring and deal-tier logic
    parsers/                Format-specific sub-parsers (nmap_xml, openvas_xml, pdf, excel)
```

```
reports/
  generator.py            CSV export

tests/
  test_triage.py          Triage unit tests

data/results/             Nmap XML output files and CSV reports
```

4. End-to-End Pipeline (Current)

When the user clicks Run Scan, the following steps execute in order:

1. Nmap scan

nmap_scan.py runs nmap -sV --open on the target. Saves raw XML to data/results/.

2. XML parsing

parser.py reads the XML. For each open port it creates a Finding object with: cvss_score=3.5 (default), exposure inferred from service name (http/ssh/rdp etc. = PARTNER, else INTERNAL), data_sensitivity=UNKNOWN, exploit_status=UNKNOWN, scanner_source=NMAP.

3. Shodan lookup

app.py resolves the target hostname to an IP (socket.gethostbyname), then calls shodan_scan.lookup_host(ip). This costs exactly 1 Shodan API credit per scan.

4. Shodan enrichment

enrich_findings_with_shodan() iterates every Finding. If the finding's port appears in Shodan's observed port list: exposure is upgraded to INTERNET_FACING, evidence_strength is set to CORRELATED (Nmap + Shodan agree), if Shodan reports any CVEs and exploit_status is still UNKNOWN it is upgraded to PUBLIC_EXPLOIT and cvss_score is floored at 6.5. Shodan context metadata (org, ASN, ISP, country, city, hostnames, domains, ports, vulns) is stored on every finding's raw_data field regardless.

5. Triage / scoring

triage_all() runs triage() on each Finding. triage() first checks override rules (deal-killer fast-paths), then calls calculate_score() which produces a 0-100 weighted score.

6. Session state

Findings, shodan_result, resolved_ip, and clean_target are stored in st.session_state so filter/slider interactions don't trigger a re-scan.

7. UI render

app.py renders: info bar, Shodan detail expander, summary metrics, filter controls (tier multiselect, exposure multiselect, score slider), findings table with ProgressColumn for risk score, detailed finding cards (toggleable), CSV download button.

5. Risk Scoring Model (triage.py)

The final risk score is a number from 0 to 100, calculated as a weighted sum of four factors and then multiplied by an evidence-strength adjustment.

Step 1 - Weighted base score

```
score = (cvss_normalised * 0.35)
      + (exposure_score  * 0.25)
      + (sensitivity_score * 0.25)
```

```
    + (exploit_score   * 0.15)
```

- CVSS (35%): Normalised from 0-10 to 0-100. Default is 3.5 (Nmap). Shodan floors it at 6.5 for matched ports with CVEs.
- Exposure (25%): INTERNET_FACING=100, PARTNER=60, INTERNAL=30, UNKNOWN=50.
- Data Sensitivity (25%): CROWN_JEWEL=100, REGULATED=85, SENSITIVE=55, LOW=20, UNKNOWN=50.
- Exploit Status (15%): ACTIVE_EXPLOITATION=100, PUBLIC_EXPLOIT=65, NO_EXPLOIT=10, UNKNOWN=30.

Step 2 - Evidence strength multiplier

The base score is multiplied by an evidence confidence factor:

- CONFIRMED (1.00) - OpenVAS / ZAP verified finding - no reduction.
- CORRELATED (0.95) - Port seen by both Nmap and Shodan - small reduction.
- INFERRED (0.85) - Shodan-only signal - larger reduction (banner-based, not verified).
- UNKNOWN (0.90) - Default for Nmap-only findings.

Step 3 - Deal-tier classification

DEAL_KILLER	Override rules fire before scoring. Any active exploitation, crown-jewel asset internet-facing with CVSS >= 9.0, regulated data internet-facing with CVSS >= 9.5, or manual active-compromise tag => score forced to 100, tier forced to DEAL_KILLER.
CRITICAL	Score >= 75.
MODERATE	Score >= 50.
MANAGEABLE	Score < 50.

6. Data Model (schema.py)

Every scanner produces Finding objects. The Finding Pydantic model is the single shared representation across the entire pipeline.

```
Finding fields:
  id                UUID (auto)
  title             Human-readable finding name
  host              IP address
  port              Integer port number
  service           Service name (http, ssh, etc.)
  cve_id            Optional CVE identifier
  cvss_score        0.0 - 10.0
  description       What was found
  remediation       How to fix it
  exposure          ExposureLevel enum
  data_sensitivity  DataSensitivity enum
  exploit_status    ExploitStatus enum
  evidence_strength EvidenceStrength enum
  scanner_source    ScannerSource enum (which scanner produced this)
  raw_data          Dict - scanner-specific metadata (Shodan context stored here)
  risk_score        Final 0-100 score (set by triage)
  deal_tier         DealTier enum (set by triage)
  override_reason   Text explanation if a deal-killer rule fired
  discovered_at     UTC timestamp
```

7. Shodan Integration Detail

Shodan is used as an enrichment layer, not a primary scanner. One api.host() call per scan (1 credit). The enrichment logic in enrich_findings_with_shodan() applies the following rules per finding:

- Shodan context (org, ASN, ISP, country, city, hostnames, domains, all observed ports, all CVEs) is written to finding.raw_data for every finding regardless of port match.
- finding.raw_data['shodan_port_match'] = True/False - records whether this specific port was seen by Shodan.
- If the port is in Shodan's observed list: exposure is upgraded to INTERNET_FACING, evidence_strength is set to CORRELATED.
- If the port matches AND Shodan reports CVEs AND exploit_status is still UNKNOWN: exploit_status is upgraded to PUBLIC_EXPLOIT, cvss_score is floored at 6.5 (conservative - Shodan CVEs are banner-based, not verified).
- Findings whose ports are NOT in Shodan's list are not penalised - they keep their Nmap-derived exposure level.
- exploit_status is only upgraded if it is currently UNKNOWN - a future OpenVAS/ZAP ACTIVE_EXPLOITATION status will never be overwritten by Shodan.

8. What Is Not Done Yet

OpenVAS integration

openvas_parse.py exists but is not connected to the pipeline. Will provide verified CVEs, real CVSS scores, and CONFIRMED evidence_strength.

ZAP integration

zap_scan.py exists but is not connected. Will provide web application findings.

CVSS from CVE IDs

Shodan returns CVE IDs but not CVSS scores. Currently all Shodan-enriched findings get cvss_score floored at 6.5. A future NVD API lookup could provide real CVSS per CVE.

data_sensitivity

All findings currently default to UNKNOWN. A future asset inventory layer (e.g. Excel upload) would classify assets as CROWN_JEWEL, REGULATED, etc., enabling the highest-impact deal-killer rules.

Multi-host scans

Subnet scans (e.g. 192.168.1.0/24) produce multiple hosts. Shodan is currently queried only for the single resolved IP of the target hostname. Multi-host Shodan lookups are not yet implemented.

9. Shodan Credit Efficiency

- api.host() = 1 credit per IP. This is the most efficient Shodan call available.
- Results are stored in st.session_state - the same scan result is reused across all filter/slider interactions without a new API call.
- Future multi-host support should batch or cache Shodan results, not query per-finding.
- Test the merge logic using saved Shodan JSON fixtures rather than live lookups during development.